

現代化 AWS 雲端開發 環境架構與管理實務

基於 VS Code + WSL + uv + AWS CDK
的隔離型高效開發工作流



標準化團隊開發環境



零環境污染與極速部署



基礎架構即程式碼 (IaC)



現代化指揮中心：Visual Studio Code

程式碼編輯器
(Code Editor)

AI 輔助側邊欄
(AI Assistant Sidebar)



開源免費與高度擴充

支援所有主流語言生態，內建 Git 版本控制，高度模組化擴充。



原生 AI 輔助支援

無縫整合 GitHub Copilot Agent，自動閱讀程式碼庫並執行重構。



企業終端零負擔 (免安裝)

支援可攜式模式 (Portable mode) 或瀏覽器 Web 版，不需本機最高管理員權限即可運行，確保主機純淨。

整合終端
(Integrated Terminal)

為什麼選擇「免安裝」的輕量化策略？

	傳統本機安裝 IDE	VS Code 免安裝 / 可攜式架構
系統影響	產生大量登錄檔 (Registry bloat)，系統日漸緩慢 	系統純淨零污染，所有設定封裝於單一資料夾 
權限與部署	需要 IT 部門核發最高管理員權限 (Admin) 才能安裝 	隨身碟或瀏覽器即可啟動，完全無須提權 
團隊一致性	成員環境設定各異，除錯困難 (It works on my machine) 	設定檔 (Settings Sync) 一鍵同步，全員開發環境 100% 標準化 

❗ 結論：開發工具不應成為 IT 管理的負擔，輕量化與免安裝是敏捷團隊的安全基石。

打造本機開發無塵室：Windows Subsystem for Linux (WSL)



無縫聯動：VS Code Remote Development



極速與統一的依賴管理：Python + uv



極致效能架構

底層由 Rust 編寫，套件解析與安裝速度比傳統 pip 提升高達 10 到 100 倍。

全域快取 (Global Cache)

智慧型依賴去重技術，相同套件無需重複下載，極大化節省主機硬碟空間與網路頻寬。

統一鎖定檔 (Lockfile)

生成具備跨平台一致性的 Universal Lockfile，確保全團隊拉取專案時，版本 100% 相同，消滅版本衝突。

在 WSL 沙盒中構築 Python 虛擬環境



Step 1. 透過 Linux 原生 curl 秒速安裝

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Step 2. 在專案目錄下建立隔離的虛擬環境

```
$ uv venv
```

Creating virtual environment at: .venv

Step 3. 毫秒級解析與同步專案依賴

```
$ uv pip sync requirements.txt
```

Resolved 43 packages in 11ms

極簡安裝：無需預先準備 Python 或 Rust，一鍵部署。

建立隔離環境：生成專屬 `.venv`，專案互不干擾。

極速同步：徹底根除全域 Python 環境污染問題。

乾淨的相依性管理：將 CDK 鎖定在沙盒中

拒絕全域污染

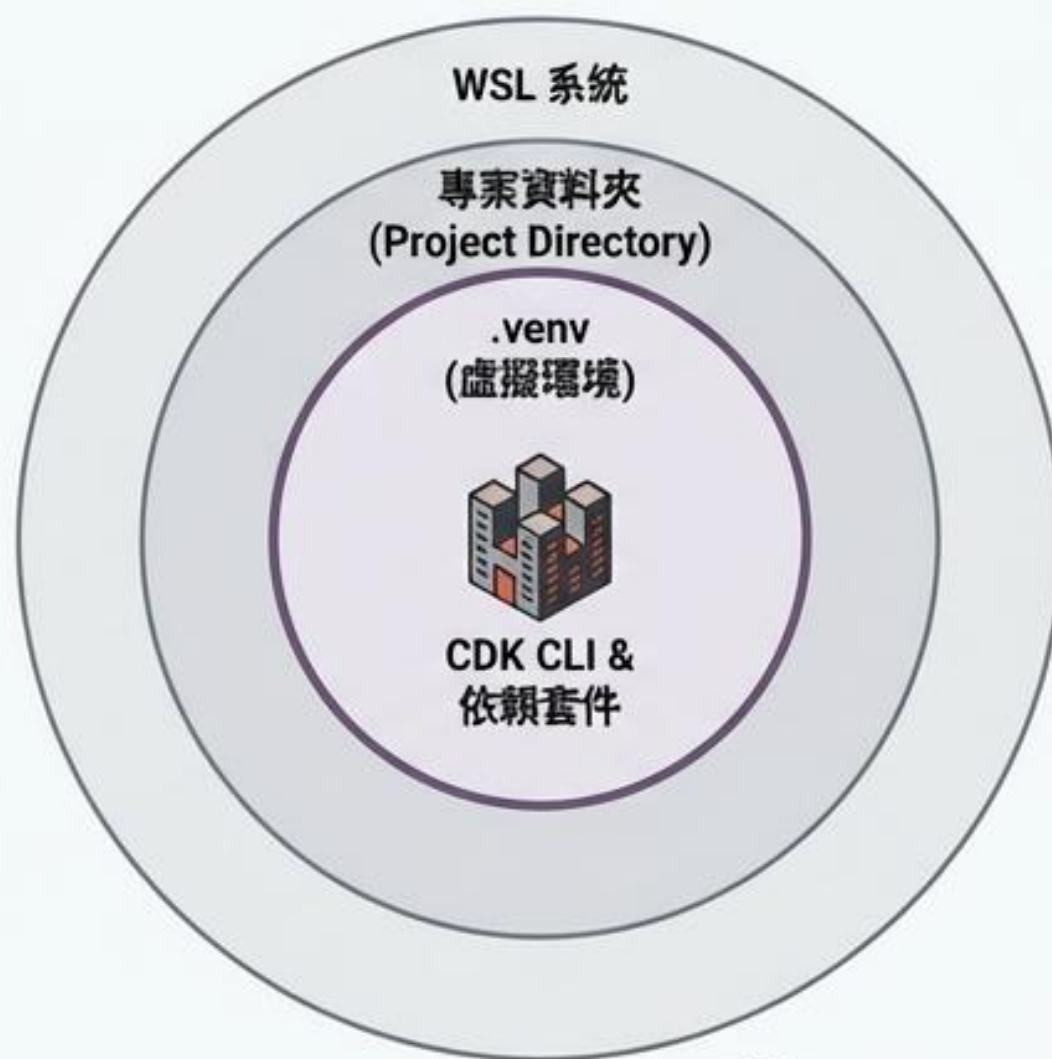
避免將套件安裝於全域環境，
徹底消除專案間的版本衝突噩夢。

專案級絕對隔離

結合 uv 與 .venv，環境隨
專案建立與銷毀，實現
100% 可重現的純淨環境。



基礎架構即程式碼：CDK CLI 虛擬化安裝

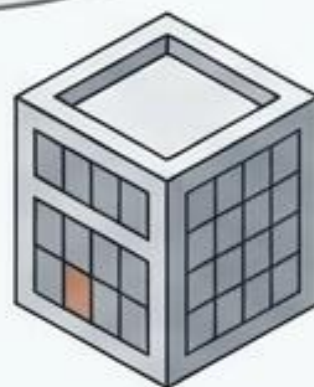


業界最佳實踐



拒絕全域安裝污染

業界標準強烈建議不要將 CDK 全域安裝，避免污染系統主環境。



絕對的依賴隔離

使用 uv 極速建立專屬的 .venv 虛擬環境，並將 CDK CLI 僅安裝於該環境內。確保多個 AWS 專案間的版本絕不衝突。

雲端自動化橋樑：AWS Command Line Interface



🔗 統一的控制中心

僅需單一 CLI 工具，即可從本機命令列深度控制並自動化成百上千種 AWS 雲端服務。

📄 取代手動風險

以可版控的指令碼 (Scripts) 取代網頁主控台 (Console) 的人工點選，大幅降低營運中的人為失誤率。

🔧 智慧工程輔助

內建自動完成 (Command completion) 功能與精確的操作歷史記錄，提升工程師開發效率。

確保生態系相容：於 WSL 內安裝 AWS CLI

```
$ curl
```

下載 Linux 專用安裝檔



```
$ unzip
```

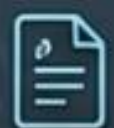
解壓縮套件內容



```
$ sudo ./install
```

執行最高權限安裝

為何堅持在 WSL 沙盒中安裝？



原生 Linux 相容性

確保 AWS CLI 完美相容未來所有的 Shell Scripts 與 CI/CD 雲端管線環境，避免路徑轉換錯誤。



零 Windows 主機依賴

完全不干擾 Windows 主機的全域環境變數，保持個人電腦的絕對乾淨。



即裝即用

遵循標準開源生態系，最低限度的組態設定即可與 AWS 服務無縫互動。

安全驗證與權限控管：AWS CLI 帳號連動



架構防護承諾：所有存取金鑰皆安全隔離於 WSL 沙盒內部，
外部 Windows 惡意程式極難竊取，徹底收攏企業雲端存取破口。

基礎架構即程式碼 (IaC) : AWS CDK 簡介



傳統營運模式 (手動管理)

工程師迷失於成百上千的網頁選單，人為設定錯率極高，難以追蹤變更。



現代化 CDK 模式 (IaC)

使用 Python 撰寫清晰的基礎架構定義，每一次架構變更皆有跡可循。



物件導向定義

Cloud Development Kit 讓開發者使用熟悉的語言操控雲端資源。



強大版本控制

伺服器架構等同於應用程式碼，可被 Git 追蹤與嚴格審查 (Code Review)。



極速災難復原

若發生災難，只需執行指令即可一鍵重建整個 AWS 雲端架構 (BCP)。

專案啟動與維護：標準化 IaC 結構

VS Code

```
my_cdk_project/  
├── app.py  
├── cdk.json  
└── infrastructure/  
    ├── vpc_stack.py  
    └── s3_bucket.py
```



一鍵初始化

使用 `cdk init app` 瞬間生成符合最佳實踐的骨架。



程式碼即架構

每一個 Python 檔案，完美對應一個真實的雲端元件。



版本控制整合

無縫搭配 Git，所有變動皆留下 Commit 紀錄便於 Review。

安全與可控的發布：三步驟部署管線 (Deployment Pipeline)



 **防呆保證：**這條流水線確保了從開發環境到雲端生產環境的每一次發布，都具備嚴謹的可追溯性與極低的營運風險。

最大化營運安全：變更差異比對 (`cdk diff` 的絕對價值)



Terminal Output Mockup

```
$ cdk diff
```

```
Stack MyDatabaseStack
```

```
Resources
```

```
[+] Added AWS::S3::Bucket MyNewBackupBucket
```

```
[~] Modified AWS::EC2::Instance WorkerNode
```

```
    Parameters: InstanceType (t3.micro -> t3.large)
```

```
[-] Destroyed AWS::RDS::DBInstance ProductionDB_critical
```

```
Security Notice: IAM Statement Changes
```

```
[+] Added: Allow s3:PutObject on MyNewBackupBucket
```



透明可視的變更

清楚揭露「哪些資源被新增、修改或刪除」。甚至連 IAM 權限的微小變更也會觸發專屬的安全警報 (Security Notice)。



決策防呆機制

徹底消滅傳統主控台上「不知道按了什麼導致服務中斷」的黑箱操作風險。每一行刪除都在掌控之中。



極高的營運信心

確保每一次部署都在團隊的預期之內，這是企業級系統進行持續交付 (Continuous Delivery) 的最重要信心基石。

結語：現代化 AWS 雲端原生開發藍圖

Ecosystem Integration Diagram



極致效能

VS Code 輕量化免安裝架構，配合 `uv` 毫秒級的 Rust 依賴解析，解放工程師生產力。



絕對安全

WSL 無塵室沙盒隔離本機風險，結合 AWS IAM 最小權限控管，鎖死資安破口。



標準化營運

CDK 程式碼化基礎架構，配合嚴格的三步驟部署管線，保證架構高可擴展與災難復原能力。

一套為現代敏捷團隊量身打造的極致工作流，全面提升交付速度與維運穩定性。

總結：現代化 IoT 雲端維運飛輪

Phase 1: 隔離防護
(VS Code + WSL)
確保底層安全，環境不干擾。

Phase 2: 極速驅動
(uv + AWS CLI)
提升 100x 解析速度，
建立安全通訊。

Phase 3: 自動部署
(CDK in .venv)
基礎設施程式碼化，變更可控。

零污染與
高效率

安全性提升 (Security)
從沙盒到虛擬環境的多重隔離，
搭配精準的憑證管理。

開發效率倍增 (Speed)
Rust 驅動的引擎與自動化 IaC
工作流，大幅縮短維運時間。

系統可維護性 (Maintainability)
所見即所得的變更預覽，讓雲端
資源更新不再是高風險操作。

部署與迭代：持續交付的工作流程



完整架構回顧：現代化 AWS 指揮中心

AWS Cloud

CDK / AWS CLI
Gateway

VS Code UI

uv Engine

uv Engine

Windows Host

1

零污染

完美的本地系統防護

3

100% 可重現

自動化、安全的
基礎架構交付

2

10x 倍速

極速 Python 環境建置